

Cell 1 — Imports and Setup (Notebook cell index: 2)

This cell loads libraries and sets random seeds so later experiments are reproducible.

Granular line-by-line breakdown

Comments and install reminder

```
# If needed, uncomment to install dependencies
# !pip install -q pandas numpy scikit-learn matplotlib seaborn nltk gensim
```

- `#` — Comment: Python ignores these lines at runtime.
 - `!pip ...` — Jupyter “magic”: run shell command **only if uncommented**. **Why:** One-shot dependency install in hosted environments.
-

Standard library imports

```
import re
```

- `import` — Loads the module namespace `re` (regular expressions).
- `re` — Module name you use as `re.sub`, etc.

```
import html
```

- `html` — Provides `html.unescape` for decoding HTML entities.

```
import random
```

- `random` — Python’s RNG module for `random.seed`, sampling.

```
import warnings
```

- `warnings` — Lets you filter or emit warning messages.

```
from collections import Counter
```

- `from collections import Counter` — Imports **only** `Counter` from `collections` into the global namespace (you write `Counter(...)`, not `collections.Counter`). **Why:** Shorter name for frequency counts.
-

Third-party: arrays, tables, plotting, word cloud

```
import numpy as np
```

- `numpy` — Numeric arrays and RNG (`np.random.seed`).
- `as np` — **Alias:** standard short name `np` instead of typing `numpy` each time.

```
import pandas as pd
```

- `pandas` — DataFrames (`pd.read_csv`, etc.).
- `as pd` — Conventional alias.

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

- **seaborn** — Statistical plots built on matplotlib; alias **sns** is convention.
- **matplotlib.pyplot** — Low-level plotting (`plt.figure`, `plt.show`); alias **plt**.

```
from wordcloud import WordCloud
```

- **WordCloud** — Class used to generate word-cloud images (not the whole wordcloud package).

NLTK

```
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
```

- **nltk** — Natural Language Toolkit; used for `nltk.download(...)`.
- **stopwords** — Function/list access like `stopwords.words('english')`.
- **PorterStemmer** / **WordNetLemmatizer** — Classes you **instantiate** later (`stemmer = PorterStemmer()`).

scikit-learn

```
from sklearn.model_selection import train_test_split, ParameterGrid
```

- **train_test_split** — Splits DataFrames/arrays into train/val pieces.
- **ParameterGrid** — Iterates dicts of hyperparameters as Cartesian product.

```
from sklearn.preprocessing import LabelEncoder
```

- **LabelEncoder** — Maps string labels to integers $0..K-1$.

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

- **TfidfVectorizer** — Turns text lists into TF-IDF sparse matrices.

```
from sklearn.linear_model import LogisticRegression
```

- **LogisticRegression** — Multiclass linear classifier.

```
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, cl
```

- Each name is a **function**: compare `y_true` vs `y_pred` with different summaries.

Gensim and TensorFlow / Keras

```
from gensim.models import Word2Vec
```

- **Word2Vec** — Class for training embedding vectors.

```
import tensorflow as tf
```

- **tf** — Root namespace for `tf.keras`, `tf.random.set_seed`, optimizers.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Embedding, SimpleRNN,
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical
```

- **Sequential** — Container that stacks layers in order.
- **Dense, Dropout, Embedding, RNN layers, Bidirectional** — Layer **classes** added with `.add(...)` or inside `Sequential([...])`.

- `EarlyStopping` — Callback passed to `model.fit`.
 - `Tokenizer` — Keras text → integer sequences.
 - `pad_sequences` — Pads/truncates sequences to fixed length.
 - `to_categorical` — Integer labels → one-hot matrices.
-

Warnings and plot style

```
warnings.filterwarnings('ignore')
```

- `warnings.filterwarnings('ignore')` — Suppresses (most) warning prints during long runs. **If omitted:** Console floods with deprecation/noise warnings.

```
sns.set_theme(style='whitegrid')
```

- `sns.set_theme` — Sets default Seaborn/matplotlib appearance (`style='whitegrid'` adds grid lines). **If omitted:** Default matplotlib styling only.
-

Random seeds

```
SEED = 42
```

- `SEED` — Integer constant reused everywhere you need reproducibility.

```
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

- `random.seed` — Seeds Python's random module (sampling, some sklearn paths).
- `np.random.seed` — Seeds NumPy's RNG (used by sklearn/gensim internals partially).
- `tf.random.set_seed` — Seeds TensorFlow/Keras operations where deterministic.

If omitted: Different train/val splits or model init across runs.

NLTK downloader loop

```
for pkg in ['punkt', 'punkt_tab', 'stopwords', 'wordnet', 'omw-1.4']:
```

- `for pkg in [...]` — Iterates string names of NLTK resources (tokenizer data, stopwords list, WordNet for lemmatizer, Open Multilingual Wordnet).

```
    try:
        nltk.download(pkg, quiet=True)
```

- `try:` — Starts block that may raise errors.
- `nltk.download(pkg, quiet=True)` — Downloads corpus pkg; `quiet=True` reduces printed chatter.

```
    except Exception as exc:
        print('nltk.download skipped or failed:', pkg, exc)
```

- `except Exception as exc` — Catches any exception; `exc` is the error object printed for debugging.
-

Final comment line

```
# After normalize_text(), tokens are letters-only words; split() avoids Pu
```



- Explains design choice in a **later** cell (simple `split()` vs NLTK sentence tokenizer).
-

Mechanism of the whole cell

Bind names to modules/classes/functions; configure warnings and plot theme; fix RNG seeds; optionally download NLTK data without crashing the notebook.

Role and importance in the notebook

Without this cell, later code raises `NameError` on imports; without seeds, comparisons across preprocessing/models are less repeatable.